

Compiler Project 2024

You are required to write a small compiler that needs to:

- Perform Syntactic Analysis for the production rules provided in this document.
- Perform Contextual Analysis, which includes:
 - Applying the scope rules – recording and referencing declarations
 - Type checking – for both commands and expressions (requires the implementation of scope rules).

Contextual Analysis considerations:

- Declaration of variables
- Have variables been declared when referred to in code
- Are the datatypes of the variables valid when used
- Inferring the data types of expressions
 - Implement operator rules here
- Assignment statements : variable and expression compatible data type
- If command : Is condition a valid boolean expression

1. Evaluation

The evaluation of your project will consist of evaluating:

- Your code partly during your booked session & also at a later stage based on your submitted files;
- Your understanding of “your” code – which will happen during the booked session by means of random questions. Examples of questions/scenarios that might occur include:
 - Asking you to debug the program and explain the various steps that occur.
 - Questions surrounding how your program caters to specific use cases.
 - How the program could be adapted to cater for a specific use case.
 - More general questions related to the compilation process.
 - Questions regarding implementation specifics, i.e., how is something implemented and why?

Assessment sessions will take place in my office (provisionally) based on a booking made with me.

Solutions can be written in Java (IntelliJ) or C# (Visual Studio). You will need to be able to set up and run your program at the start of the assessment session. Files need to be submitted on Moodle at least 1 day before your assessment session.

2. Assessment

A rough idea of how you will be assessed:

- Assessment session: 40%
- Correct functioning of code: 20%
- Adhering to general coding principles: 10%
- Extensibility (i.e., can your code easily cater to new production rules): 10%
- Use of appropriate data structures: 10%
- Readability: 10%

3. Deadline

25 November 2024

4. Production Rules

Program	::=	Command
Command	::=	if Expression then Command else Command V-name := Expression let Declaration in Command
Expression	::=	PrimaryExpression Operator PrimaryExpression
PrimaryExpression	::=	V-name (Expression) Int-literal
V-name	::=	a b c d e
Operator	::=	+ - * / < > =
Declaration	::=	Single-Declaration Declaration ; Declaration
Single-Declaration	::=	var V-name : Type-denoter
Type-denoter	::=	int double boolean
Int-literal	::=	1 2 3

5. Code Template

Courtesy of Prof Greyling

Program class

```
class Program
{
    static void Main(string[] args)
    {
        Parser P = new Parser("a + ( b - c )");
        Console.ReadLine();
    }
}
```

Token class

```
class Token
{
    String Spelling;
    int Type;          // 1 - Identifier, 2 - Operator, 3 - LPar, 4 - RPar
    public Token(String S, int T)
    {
        Spelling = S;
        Type = T;
    }

    public String getSpelling()
    {
        return Spelling;
    }

    public int getType()
    {
        return Type;
    }

    public void showSpelling()
    {
        Console.WriteLine(Spelling);
    }

    public Boolean matches(String other)
    {
        return (this.Spelling.Equals(other));
    }

    public Boolean matchesType(int T)
    {
        return (Type == T);
    }
}
```

Scanner class

```
class Scanner
{
    const int Identifier = 1;
    const int Operator = 2;
    const int LPar = 3;
    const int RPar = 4;

    ArrayList TokenList = new ArrayList();
    String Sentence;
    int curPos;

    public Scanner(String S)
    {
        Sentence = S;
        BuildTokenList();
        curPos = 0;
    }

    public void DisplayTokens()
    {
        for (int x = 0; x <= TokenList.Count - 1; x++)
            ((Token)TokenList[x]).showSpelling();
    }

    public ArrayList getTokens()
    {
        return TokenList;
    }

    String BuildNextToken()
    {
        String Token = "";
        while (Sentence[curPos] == ' ')
            curPos++;
        while ((curPos < Sentence.Length) && (Sentence[curPos] != ' '))
        {
            Token = Token + Sentence[curPos]; curPos++;
        }
        return Token;
    }

    int FindType(String Spelling)
    {
        if (Spelling.Equals("("))
            return LPar;
        if (Spelling.Equals(")")
            return RPar;
        if (Spelling.Equals("+"))
```

```
        return Operator;
    if (Spelling.Equals("-"))
        return Operator;
    if (Spelling.Equals("*"))
        return Operator;
    if (Spelling.Equals("/"))
        return Operator;
    else
        return Identifier;
}

void BuildTokenList()
{
    Token newOne = null;
    while (curPos < Sentence.Length)
    {
        String nextToken = BuildNextToken();
        newOne = new Token(nextToken, FindType(nextToken));
        TokenList.Add(newOne);
    }
}
}
```

Parser class

```
class Parser
{
    ArrayList TokenList; Token CurrentToken; int CurTokenPos;

    const int Identifier = 1;
    const int Operator = 2;
    const int LPar = 3;
    const int RPar = 4;

    public Parser(String Sentence)
    {
        Scanner S = new Scanner(Sentence);

        TokenList = S.getTokens();
        CurTokenPos = -1;
        FetchNextToken();
        Expression P = parseExpression();
    }

    void FetchNextToken()
    {
        CurTokenPos++;
        if (CurTokenPos < TokenList.Count)
            CurrentToken = (Token)TokenList[CurTokenPos];
        else
            CurrentToken = null;
    }

    void accept(int Type)
    {
        if (CurrentToken.matchesType(Type))
            FetchNextToken();
        else
            Console.WriteLine("Syntax Error in accept");
    }

    void acceptIt()
    {
        FetchNextToken();
    }

    Expression parseExpression()
    {
        Expression ExpAST;
        PrimaryExpression P1 = parsePrimary();
        Operate O = parseOperator();
    }
}
```

```

        PrimaryExpression P2 = parsePrimary();
        ExpAST = new Expression(P1, 0, P2);
        return ExpAST;
    }

    PrimaryExpression parsePrimary()
    {
        PrimaryExpression PE;
        if (CurrentToken == null)
            return null;
        switch (CurrentToken.getType())
        {
            case Identifier: Identifier I = parseIdentifier();
                PE = new IdentifierPE(I);
                break;
            case LPar: acceptIt();
                PE = new BracketsPE(parseExpression()); accept(RPar);
                break;
            default: Console.WriteLine("Syntax Error in Primary");
                PE = null;
                break;
        }
        return PE;
    }

    Identifier parseIdentifier()
    {
        Identifier I = new Identifier(CurrentToken.getSpelling());
        accept(Identifier);
        return I;
    }

    Operate parseOperator()
    {
        Operate O = new Operate(CurrentToken.getSpelling());
        accept(Operator);
        return O;
    }
}

```

AST class

```
abstract class AST
{
}

public class Expression : AST
{
    public PrimaryExpression P1; public Operate O;
    public PrimaryExpression P2;

    public Expression(PrimaryExpression P1, Operate O, PrimaryExpression P2)
    {
        this.P1 = P1;
        this.O = O;
        this.P2 = P2;
    }
}

public class PrimaryExpression : AST
{
}

public class IdentifierPE : PrimaryExpression
{
    Terminal T;

    public IdentifierPE(Terminal T)
    {
        this.T = T;
    }
}

public class BracketsPE : PrimaryExpression
{
    Expression E;

    public BracketsPE(Expression E)
    {
        this.E = E;
    }
}

public class Terminal : AST
{
    String Spelling;

    public Terminal(String Spelling)
```



```
{
    this.Spelling = Spelling;
}

public class Identifier : Terminal
{
    public Identifier(String Spelling) : base(Spelling)
    {
    }
}

public class Operate : Terminal
{
    public Operate(String Spelling) : base(Spelling)
    {
    }
}
```